IEAI-ALP

ERP Access Intelligence & Adaptive Least Privilege

Computer Architecture and Organization project report

Abstract

IEAI-ALP is an access-governance research application with a web frontend, Python API, role-mining engine, approval-backed policy changes, audit history, and an AWS serverless deployment option. Five published access-control matrices provide real evaluation inputs. The project preserves the original anonymized assignments and labels six extra CSV fields as computed graph features, never as original employee or ERP attributes.

Problem and boundaries

Flat user-to-permission grants are difficult to inspect and maintain. The application groups access profiles, discovers candidate roles, measures representation cost, highlights structurally unusual grants, and lets an administrator review, revoke, or restore a grant with policy-version checks. These are benchmark-policy experiments. They do not provision a live ERP, prove business-role correctness, identify threats, or establish patentability.

Datasets and provenance

The test_sets directory contains Americas Small, APJ, Healthcare, Domino, and Firewall2. These are distinct real-world access matrices in the HP Labs role-mining benchmark collection, representing separate source systems/domains; they are not five time partitions. The raw two-column source files are retained alongside import-ready CSVs. Each CSV has the source user and permission IDs plus six deterministic statistics calculated only from that matrix. A builder verifies published dimensions and records SHA-256 hashes in manifest.json. Dataset source links, counts, feature formulas and attribution notes are in test_sets/README.md.

Feature columns

The additional columns are derived_user_assignment_count, derived_permission_user_count, derived_user_access_density, derived_permission_prevalence, derived_user_degree_percentile, and derived_permission_degree_percentile. They are computed from the same source edge list, validated at ingestion, and recomputed for application analytics. They help describe grant volume, rarity and prevalence. They are not demographics, timestamps, approval outcomes, actions, or labels.

Role discovery and optimization

The engine starts from exact user-permission assignments, groups identical access sets, factors shared permission support signatures using a bounded heuristic, and compares direct, cohort and factored representations using unit WSC: roles plus user-role assignments plus role-permission assignments. It checks that the selected representation reconstructs every original grant without missing or extra grants. This check establishes exact reconstruction for the chosen result, not global optimality.

User profiling and risk review

Profiles show observed assignment counts, permission density, rare-grant counts, mined memberships and degree percentiles. Permission catalogs show actual support/prevalence. Review scores use explicitly

documented structural features and are cues for a person to inspect; no threat labels or business impact are inferred. Since these benchmark matrices are static, temporal behavior and approval-delay prediction are unavailable.

Policy changes and reinforcement learning

Administrators can retain, revoke or restore grants. Expected-version checks prevent stale changes; parent versions, actors and audit events record outcomes. Authorization is checked against the active mined policy. Offline terminal Q-learning is limited to actual administrator feedback, requires at least five genuine reviews across two users, and abstains when evidence is insufficient. No synthetic reviews, rewards, or benchmark-based temporal outcomes are included.

Cloud architecture

The SAM/CloudFormation definition configures API Gateway, Lambda, Cognito, private S3, DynamoDB, IAM and CloudWatch Logs; AWS Budgets provides alerts and Access Analyzer validates IAM policy documents. S3 stores uploaded source files and computed artifacts, Lambda runs the API and analysis, Cognito authenticates the administrator, and DynamoDB stores metadata and policy/audit state. The browser can remain local. Deployment and usage depend on the account; free-tier credits are not guaranteed and a budget alert does not cap charges.

Reproducibility

Run python scripts/build_test_sets.py to rebuild the five CSVs and manifest from the retained raw matrices. Import a CSV through the local application or AWS upload workflow. The manifest records source hashes and expected dimensions. The source data collection and repository are cited in test_sets/README.md. Review dataset terms before redistribution; the mirror does not state a separate dataset license.

Portfolio positioning

This project demonstrates data provenance, feature engineering, sparse access analytics, role discovery, API/backend design, policy governance, auditability, and serverless AWS integration. A defensible CV description should state the exact benchmark names and that the extra CSV fields are derived. Novelty or patentability requires separate prior-art and legal review.

Dataset dimensions

Benchmark	Users	Permissions	Assignments
Americas Small	3,477	1,587	105,205
Apj	2,044	1,164	6,841
Healthcare	46	46	1,486
Domino	79	231	730
Firewall2	325	590	36,428

References

HP Labs report

Benchmark matrix repository

Published benchmark dimensions

AWS SAM